TP 1.5 : Exercices supplémentaires

Programmation PL/pgSQL, procédures, triggers et concurrence

Module : Bases de Données Avancées
Niveau : M1
Responsable du TP : M. Alachaher
Année universitaire : 2026/2027

Objectif

Approfondir la programmation des procédures et des triggers à travers des situations bancaires plus complexes. Ces exercices sont facultatifs et ne remplacent pas les exercices du TP 1.

Préparation

Utiliser le même fichier `dataset_tp1.sql` et le schéma `tp1_banque`. **Ne pas réinitialiser le dataset** si vous souhaitez conserver vos procédures et triggers du TP 1. Dans une nouvelle fenêtre du Query Tool :

```
SET search_path TO tp1_banque, public;
SELECT id_compte, type_compte, solde, actif FROM compte
ORDER BY id_compte;
```

Remarque

Les exercices de ce complément sont indépendants autant que possible. Pour les essais qui modifient les soldes, relever les valeurs initiales et prévoir des tests avec `BEGIN` / `ROLLBACK` afin de ne pas perturber les exercices suivants. Les règles déjà créées par les triggers du TP 1 restent actives.

Partie A — Procédures stockées : exercices supplémentaires

Exercice 1 — Retrait sécurisé

1. Créer une procédure `effectuer_retrait(p_compte INTEGER, p_montant NUMERIC)` qui débite le compte indiqué.
2. Refuser un montant nul ou négatif, un compte inexistant, inactif ou dont le solde est insuffisant.
3. Enregistrer le retrait dans `operation` uniquement si le débit a réussi.
4. Tester un retrait de 2 000 DA du compte 106, puis une tentative de 10 000 DA. Vérifier le solde et le journal après chaque appel.

Exercice 2 — Prime sur les comptes d'épargne

1. Créer une procédure `appliquer_prime_epargne(p_taux NUMERIC)` qui augmente de ce pourcentage le solde de tous les comptes d'épargne actifs.
2. Refuser un taux inférieur ou égal à zéro, ou supérieur à 5.
3. Utiliser `GET DIAGNOSTICS ... = ROW_COUNT` pour afficher le nombre de comptes mis à jour.
4. Tester avec un taux de 2 % et expliquer pourquoi le compte 108 ne doit pas être modifié.

Exercice 3 — Virement avec contrôle de cohérence

1. Créer une procédure `virement_controle(p_source INTEGER, p_dest INTEGER, p_montant NUMERIC)` qui effectue un virement et enregistre deux opérations.
2. Ajouter un contrôle empêchant le virement entre deux comptes identiques ou vers un compte inactif.
3. Lever une exception si une vérification échoue, afin qu'aucune modification partielle ne soit conservée.
4. Tester un virement valide, puis un virement vers le compte 108. Expliquer l'effet de l'exception sur les mises à jour.

Partie B — Triggers : exercices supplémentaires

Exercice 4 — Contrôler la date d'une opération

1. Créer une fonction trigger qui refuse l'insertion d'une opération dont la date est postérieure à l'instant courant.
2. Associer cette fonction à un trigger `BEFORE INSERT` sur `operation`.
3. Tester une opération datée de demain et une opération datée d'hier.
4. Expliquer pourquoi ce contrôle doit être appliqué avant l'insertion.

Exercice 5 — Journaliser la désactivation d'un compte

1. Créer une table `historique_statut` contenant un identifiant, le compte concerné, l'ancien statut, le nouveau statut et la date du changement.
2. Créer une fonction trigger enregistrant les changements du champ `actif` dans cette table.
3. Créer le trigger correspondant et tester la désactivation du compte 107, puis sa réactivation.
4. Exécuter une mise à jour qui conserve le même statut : faut-il ajouter une ligne d'historique ? Adapter le trigger si nécessaire.

Exercice 6 — Limiter les opérations sur un compte inactif

1. Créer une fonction trigger qui empêche l'insertion d'une nouvelle opération dans `operation` si le compte associé est inactif.
2. Associer cette fonction à un trigger `BEFORE INSERT`.
3. Tester une opération sur les comptes 108 et 101, puis expliquer les différences observées.
4. **Réflexion :** En quoi ce contrôle est-il différent du trigger du TP 1 qui empêche une diminution de solde sur un compte inactif ? Les deux sont-ils complémentaires ?

Remarque

Ne pas supprimer les triggers du TP 1 pour réussir les exercices supplémentaires : les éventuelles interactions entre règles font partie de l'apprentissage. En cas de conflit, identifier le trigger responsable et expliquer son comportement.